Predictive Cache-Aware CI Scheduler

Company inspiration: Blacksmith | End-to-end build instructions + copy/paste coding-agent prompts

01 | Product brief

Problem: Build jobs queue up, cold-start slowly and waste compute/cache resources.

Build: Simulated GitHub Actions-compatible scheduler with runner pools, warm capacity and locality-aware assignment.

Audience: engineering teams and technical evaluators. Deliverable: working local MVP, reproducible tests, benchmark report, short demo and documented limitations.

This is an independent proof-of-work proposal. It does not assert knowledge of the company's private architecture or current incidents.

02 | Recommended architecture and repository

Java 21 + Spring Boot 3 for APIs/orchestration, React + TypeScript for dashboard, PostgreSQL for state, Redis for short-lived state, Kafka where asynchronous events are justified, Docker Compose locally, AWS/Kubernetes later. Substitute Python services where ML/browser libraries require it.

Monorepo: /apps/web, /services/api, /services/worker, /packages/contracts, /infra/docker, /tests/e2e, /benchmarks, /docs. Use OpenAPI, structured logs, OpenTelemetry, health checks and environment-variable configuration.

03 | Build milestones (execute in order)

Phase 0 - Contract and safety — Write README, requirements, threat model, architecture diagram, non-goals and measurable acceptance criteria. Create repository, .gitignore, CI and coding conventions.

Phase 1 - Working skeleton — Bootstrap backend, frontend, PostgreSQL and local Docker Compose. Add health endpoints, migrations, request validation and one end-to-end smoke test.

Phase 2 - Core data and APIs — Implement domain entities, API contracts, permissions, idempotency, state transitions and integration tests.

Phase 3 - Functional engine — Implement the domain-specific modules below, one by one, with unit tests and failure handling.

Phase 4 - Agent and automation — Introduce optional coding/AI agents only behind explicit interfaces, limits, trace logging and deterministic fallbacks.

Phase 5 - Reliability and observability — Add retries, dead-letter handling where needed, timeouts, rate limits, dashboards, tracing and chaos/failure scenarios.

Phase 6 - Evaluation and packaging — Run benchmarks, write limitations, record demo, prepare deployment guide and clean sample dataset.

04 | Functional modules - implementation checklist

- Ingest synthetic CI job arrivals and workload metadata
- Runner inventory, warm/cold states and lease lifecycle
- Content-addressed cache metadata and locality scoring
- Priority/fairness scheduling with capacity and SLA constraints
- Failure injection, recovery and benchmark simulator

05 | API, persistence and events

Representative API routes: POST /jobs, GET /jobs/{id}, GET /runners, GET /metrics

Core tables: jobs, runners, runner_leases, cache_entries, schedule_decisions

Async events (only if needed): job.queued, runner.ready, job.started, job.finished, runner.failed

Design rules: every mutation validates input and permissions; writes are idempotent where retried; DB migrations are reversible where practical; event consumers are at-least-once safe; every state transition is observable.

06 | Copy/paste agent prompt: start from zero

```
You are the lead staff engineer and implementation agent for an ORIGINAL prototype called "Predictive
Cache-Aware CI Scheduler" inspired by the public product area of Blacksmith. Do not imply affiliation or access
to private company systems.
MISSION: Build jobs queue up, cold-start slowly and waste compute/cache resources.
BUILD: Simulated GitHub Actions-compatible scheduler with runner pools, warm capacity and locality-aware
assignment.
STACK: Java 21 + Spring Boot 3 for APIs/orchestration, React + TypeScript for dashboard, PostgreSQL for state,
Redis for short-lived state, Kafka where asynchronous events are justified, Docker Compose locally,
AWS/Kubernetes later. Substitute Python services where ML/browser libraries require it.
FIRST: inspect the repository. If empty, scaffold the monorepo. Write docs/PRD.md, docs/ARCHITECTURE.md,
docs/THREAT_MODEL.md, docs/PLAN.md, and docs/DECISIONS.md. Specify user stories, non-goals, interfaces, threat
model, and measurable acceptance tests.
THEN: implement in small vertical slices. After every slice, run formatting, compilation, unit tests, and smoke
tests. Fix failures before proceeding. Commit logically and update docs/PLAN.md with completed work.
MANDATORY MODULES: Ingest synthetic CI job arrivals and workload metadata; Runner inventory, warm/cold states
and lease lifecycle; Content-addressed cache metadata and locality scoring; Priority/fairness scheduling with
capacity and SLA constraints; Failure injection, recovery and benchmark simulator.
CONTRACTS: POST /jobs, GET /jobs/{id}, GET /runners, GET /metrics. TABLES: jobs, runners, runner_leases,
cache_entries, schedule_decisions. EVENTS: job.queued, runner.ready, job.started, job.finished, runner.failed.
CONSTRAINTS: Run untrusted CI code only in hardened isolated workers; MVP uses simulated execution by default.
Never invent benchmark results or claim production readiness. Ask for approval before destructive actions, cloud
spending, deploying, or accessing real user data. Provide exact commands to run locally. Start by writing the
plan and scaffolding Phase 0 and Phase 1.
```

07 | Copy/paste agent prompt: implement core engine

```
Continue the Predictive Cache-Aware CI Scheduler repository from the current state. Read docs/PLAN.md and tests
before changing code. Implement the NEXT unfinished domain module, prioritizing: Ingest synthetic CI job
arrivals and workload metadata; Runner inventory, warm/cold states and lease lifecycle; Content-addressed cache
metadata and locality scoring; Priority/fairness scheduling with capacity and SLA constraints; Failure
injection, recovery and benchmark simulator. Add schemas, migrations, API validation, permissions, retry
semantics, structured logging, unit tests and at least one end-to-end happy-path and failure-path test. Keep
changes reviewable; do not skip tests or silently change architecture. At the end report changed files, exact
run commands, test results, unresolved risks and next step.
```

08 | Copy/paste agent prompt: testing, evaluation and hardening

```
Act as an independent QA, security and performance engineer for Predictive Cache-Aware CI Scheduler. Do not
assume the implementation works. Read the PRD, inspect code and execute tests. Create reproducible fixtures and
benchmarks for: Compare FIFO against locality-aware policy on reproducible trace; Measure p95 queue delay and
cache hit rate; Quantify cost/job and fairness across tenants. Test timeouts, retries, duplicate events, partial
failures, permissions, injection risks, data leakage and worker crashes as applicable. Record real observed
results, not fabricated numbers. Create docs/BENCHMARKS.md and docs/SECURITY_REVIEW.md with reproducible
commands and prioritized fixes. Implement the highest-severity fixes and rerun tests. Enforce: Run untrusted CI
code only in hardened isolated workers; MVP uses simulated execution by default.
```

09 | Copy/paste agent prompt: demo and deployment

```
Prepare Predictive Cache-Aware CI Scheduler as a public engineering proof-of-work. Add sample synthetic data,
one-command local setup, seed script, health checks, demo walkthrough, architecture diagram, API documentation
and troubleshooting. Add a 3-minute demo script that shows the problem, core workflow, failure recovery and
measured results. Provide optional AWS deployment architecture and rough cost assumptions, but do not provision
cloud resources without explicit approval. Ensure README distinguishes implemented features from future ideas.
```

10 | Definition of done and benchmark plan

• Compare FIFO against locality-aware policy on reproducible trace

• Measure p95 queue delay and cache hit rate

• Quantify cost/job and fairness across tenants

• Fresh clone can start locally using documented commands.

• Unit, integration and end-to-end tests pass in CI.

• Security restrictions and human approval boundaries are documented and tested.

• Benchmark numbers are reproduced from scripts and committed sample fixtures.

• README includes limitations, architecture trade-offs and a short product demo.

11 | Suggested four-sprint execution

Sprint 1: PRD, scaffolding, domain model, API skeleton and local stack.

Sprint 2: core modules, happy-path integration, synthetic data and dashboard.

Sprint 3: failure handling, security boundaries, telemetry and agent adapters.

Sprint 4: benchmarking, documentation, polish and demo recording.

12 | Next-level research directions

After the MVP works, identify the highest-cost or lowest-reliability component from measured data. Compare a simple baseline against one optimized algorithm. Publish an experiment with dataset, configuration, confidence intervals where meaningful, and limitations. Avoid unvalidated performance claims.